

Financial Maze 3D

Project Documentation & Complete Interview Preparation Guide

React 18

React Three Fiber

Three.js

JavaScript

Vite

Context + useReducer

Web Audio API

localStorage

An interactive 3D browser game that teaches Indian personal finance

Repository: github.com/kavya-tantuvay/Financial_Maze

Contents

1. Project Overview
2. Architecture & File Structure
3. Key Technical Decisions
4. Interview Q&A — Project Overview & Motivation
5. Interview Q&A — React Fundamentals
6. Interview Q&A — State Management
7. Interview Q&A — React Three Fiber & 3D Graphics
8. Interview Q&A — Game Logic, Maze & Collision
9. Interview Q&A — Performance & Optimisation
10. Interview Q&A — Audio, Input & Accessibility
11. Interview Q&A — Build, Tooling & Deployment
12. Interview Q&A — Testing & Debugging (Real Bugs I Fixed)
13. Interview Q&A — Financial Domain Knowledge
14. Interview Q&A — Design Decisions & Trade-offs
15. Interview Q&A — Behavioural & "Tell Me About This Project"
16. Known Limitations & Future Work
17. Quick Revision Cheat Sheet

1. Project Overview

1.1 What it is

Financial Maze 3D is a browser game where the player moves a glowing sphere through a neon 3D maze. At each junction there is a glowing gold orb. Walking into an orb freezes movement and opens a financial decision scenario written for the Indian context — first salary, EPF, term insurance, home loans, the old versus new tax regime. The player picks an option, and the game immediately shows how that choice moved three scores and explains, in two or three lines, *why* it was smart or expensive.

The exit portal of each level stays locked until every decision in that level has been answered, so the player cannot sprint past the educational content.

1.2 The problem it solves

Financial literacy content in India is mostly text — blog posts, PDFs, YouTube explainers. It is passive, and none of it lets you feel the consequence of a bad decision. Someone at 22 does not know that a "no cost EMI" hides the interest in the forgone discount, or that refusing EPF throws away a guaranteed employer match. This project turns those lessons into choices with visible consequences, which is a far stickier way to learn.

1.3 Target user

An 18–25 year old in India who has just started earning or is about to. The three levels map onto the first decade of a career.

Level	Name	Situation	Income
1	First Salary	Fresher, first paycheck	₹25,000/month
2	Growing Income	Two years in, one job switch	₹50,000/month
3	Life Decisions	Home loan, child, ageing parents	₹1,00,000/month

1.4 The three scores

Score	What it measures	Why it exists
Wealth	Long-term growth — SIPs, equity, compounding	Rewards investing early
Security	Emergency fund, health cover, term insurance	A high-return portfolio with no safety net is fragile
Lifestyle	Whether the plan is liveable	A plan you resent is a plan you abandon in four months

Having three axes rather than one score is the core design idea. It means there is rarely a single "correct" answer — maximising Wealth by locking every rupee into VPF costs Lifestyle and liquidity. That mirrors how real financial planning actually works, and it makes the game worth replaying.

1.5 Project statistics

Source files	31 (excluding config and lockfile)
Levels	3, maze sizes 17×17, 19×21, 23×23 cells
Scenarios	18 total, 6 per level, 3–4 options each
Glossary terms	18
npm dependencies	187 packages installed, 6 direct
Production bundle (gzipped)	~176 KB three.js + ~45 KB React + ~68 KB app + ~4 KB CSS
Backend	None. No server, no database, no authentication.

2. Architecture & File Structure

The application has three clearly separated layers, and the separation is deliberate: **content** (what the game teaches), **rules** (how state changes), and **presentation** (3D scene and 2D overlay).

```

src/
├─ main.jsx           Mounts the app inside <GameProvider>
├─ App.jsx            Canvas + UI overlay; decides which screen shows
├─ index.css          All styling for the 2D interface
├─
├─ data/              — CONTENT LAYER (no logic) —
│   ├── scenarios.js  18 scenarios: options, score impacts, explanations
│   ├── levels.js     3 maze layouts as text art + level metadata
│   └─ glossary.js    18 financial terms in plain English
├─ context/
│   └─ GameContext.js — RULES LAYER — Context + useReducer + autosave
├─ hooks/
│   ├── useControls.js Keyboard + joystick input
│   └─ useSound.js     Web Audio effects and ambient pad
├─ three/              — 3D PRESENTATION —
│   ├── Scene.jsx     Lights, fog, stars; assembles the scene
│   ├── Maze.jsx       Instanced glowing walls
│   ├── Floor.jsx      Dark metallic floor + glowing grid
│   ├── Player.jsx     Player sphere, movement, collision, triggers
│   ├── DecisionNode.jsx Pulsing gold orbs with orbiting particles
│   ├── ExitPortal.jsx Locked / unlocked exit ring
│   └─ FollowCamera.jsx Smooth follow camera
├─ ui/                 — 2D PRESENTATION —
│   ├── StartScreen.jsx Title screen and level select
│   ├── HUD.jsx         Top bar: level, score bars, balance
│   ├── ScoreBar.jsx    One animated score bar
│   ├── ScenarioPopup.jsx The decision card
│   ├── FeedbackCard.jsx "Why this was good or bad" card
│   ├── LevelComplete.jsx Level summary and grade
│   ├── FinalReport.jsx Financial health report after Level 3
│   ├── FinancialGlossary.jsx Slide-out glossary panel
│   └─ Joystick.jsx     Touch control for mobile
└─ utils/
    ├── format.js      ₹ formatting, grading, compound interest
    └─ maze.js         Text grid → 3D walls, collision, movement

```

2.1 Data flow

```
Player walks into an orb (Player.jsx, inside useFrame)
|
▼ onEnterNode(node)
App.jsx → dispatch({ type: "OPEN_SCENARIO", scenario })
|
▼
GameContext reducer → phase = SCENARIO (movement freezes)
|
▼
ScenarioPopup renders; player clicks an option
|
▼ dispatch({ type: "CHOOSE_OPTION", scenario, option })
Reducer applies impact to scores, money to balance, records the decision
|
▼
phase = FEEDBACK → FeedbackCard explains why
|
▼ dispatch({ type: "CLOSE_FEEDBACK" })
phase = PLAYING (movement resumes)
```

2.2 The phase state machine

A single `phase` field decides which overlay renders and whether the player can move. This avoids a tangle of independent booleans like `isPopupOpen`, `isPaused`, `showResults` that can contradict each other.

```
START → PLAYING ⇌ SCENARIO → FEEDBACK → PLAYING
      |
      ↓
      LEVEL_COMPLETE → PLAYING (next level)
      |
      ↓
      GAME_COMPLETE
```

3. Key Technical Decisions

Decision	Reasoning
Vite instead of Create React App	CRA is deprecated and breaks on Node 24 with React 18/19. Vite's <code>start</code> script was aliased so <code>npm start</code> still works as expected.
Context + useReducer, not Redux	One reducer, eight action types, no async, no middleware. Redux would be ceremony without benefit — and this is a learning project I need to explain.
Web Audio API, not Howler.js	Sounds are synthesised in code, so there are zero audio files to download, no licensing questions, and no extra library in the bundle.
Mazes as text art	A whole level can be redesigned by editing strings — no 3D code changes.
All content in one data file	<code>scenarios.js</code> is readable top to bottom. Content and code are separable, so a non-programmer could review the financial advice.
Instanced meshes for walls	200+ walls in one draw call instead of 200 draw calls per frame.
Position in a ref, not state	The player moves 60 times a second; state would trigger 60 React renders per second across the whole tree.

4. Interview Q&A — Project Overview & Motivation

Q Tell me about this project in two minutes.

A Financial Maze 3D is an educational browser game built with React and React Three Fiber. The player navigates a 3D neon maze, and at each junction a real Indian financial decision appears — what to do with your first ₹25,000 salary, whether to buy a ₹1 crore term plan, old versus new tax regime. Each choice moves three scores (Wealth, Security, Lifestyle) and shows an explanation of why it was good or bad. There are three levels representing ₹25k, ₹50k and ₹1 lakh monthly income, with 18 scenarios total. Technically it is a pure front-end app: React 18, React Three Fiber for the 3D, Context plus useReducer for state, and localStorage for saving progress. No backend at all. The interesting engineering problems were rendering 200+ maze walls efficiently using instanced meshes, running the game loop through `useFrame` without triggering React re-renders, and writing collision detection that lets the player slide along walls instead of sticking to them.

Q Why did you build this? What problem does it solve?

A Financial literacy content in India is almost entirely passive — articles and videos. You read that an emergency fund matters, you nod, and you do nothing. A game makes the consequence visible: you choose the iPhone EMI, your Security score drops 10 points and the game tells you that you have just committed 26% of your salary for two years to a depreciating asset. I wanted to build something that was genuinely useful, not just a portfolio piece, and 3D made it something people would actually open.

Q Who is the target user, and how did that shape the design?

A An 18–25 year old Indian who has just started earning. That shaped several things: every amount is a rupee amount with Indian digit grouping (₹12,34,567, not \$1,234,567); the scenarios use products Indians actually have — EPF, PPF, ELSS, NPS, Sukanya Samriddhi; the glossary explains jargon in plain English because a fresher may not know what ELSS means; and the tone avoids shaming, because "spending some of your bonus is fine" is more likely to be followed than "never spend anything".

Q Why three scores instead of one?

A Because real financial planning is a trade-off, not a leaderboard. If there were one score, every scenario would have one right answer and the game would be a quiz. With three axes, locking every rupee into VPF maximises Security but tanks Lifestyle and liquidity, and the game says so. It teaches balance, which is the actual lesson, and it makes replaying worthwhile.

Q How is this different from a quiz app?

A Three ways. First, there is no single correct answer for most scenarios — several options are marked "good" for different reasons. Second, choices are cumulative: your balance and scores carry across levels, so the game builds a portfolio narrative rather than scoring isolated questions. Third, the final report converts your decisions into a real compound interest projection — "if you keep investing ₹11,000/month at 12%, you will have ₹9.1 lakh by 30" — which a quiz cannot do.

Q How long did it take, and what was the hardest part?

A The hardest part was not the 3D — React Three Fiber makes that surprisingly approachable. It was the content. Writing 18 scenarios with 3–4 options each, where every option has an honest explanation and a defensible score impact, meant checking real rates: EPF at 8.25%, PPF at 7.1%, home loans at 8.5%, education inflation at 8%. Getting those numbers right and internally consistent took longer than the code.

5. Interview Q&A — React Fundamentals

Q Which React hooks did you use and why?

- A**
- `useState` — small local UI state, like whether the glossary panel is open and the glossary search text.
 - `useReducer` — the entire game state, because transitions are complex and interdependent.
 - `useContext` — so any component can read game state without prop drilling through the 3D scene.
 - `useRef` — the player position, held keys, and Three.js object handles. Values that change every frame but must not trigger renders.
 - `useEffect` — registering keyboard listeners, autosaving to localStorage, resetting the player when the level changes.
 - `useLayoutEffect` — writing instanced mesh matrices before the first paint.
 - `useMemo` — caching the parsed maze so it is not recomputed on every render.
 - `useCallback` — stabilising the event handlers passed down into the 3D scene.

Q Why `useLayoutEffect` instead of `useEffect` in Maze.jsx?

- A** Instanced meshes start with all their instance matrices at the identity, which puts every wall at the origin. `useEffect` runs *after* the browser paints, so you would see a single frame where all 200 walls are stacked on top of each other at the centre of the maze before they jump into place. `useLayoutEffect` runs synchronously after the DOM mutation but before paint, so the matrices are correct on the very first visible frame. It is the standard fix for "flash of wrong layout".

Q Why is the player position in a `useRef` and not `useState` ?

- A** Because it changes 60 times per second. With `useState`, every frame would call a setter, which schedules a re-render of the component and everything below it — the whole 3D scene, the HUD, all of it. At 60fps that is 60 full React reconciliations per second, which would drop frames badly. A ref is a mutable box: writing to it does not notify React at all. The Three.js objects are mutated directly inside `useFrame`, which is exactly the pattern React Three Fiber is designed around — React builds the scene graph once, then the animation loop mutates it imperatively.

Q How do the camera and the player stay in sync if the position is in a ref?

- A** `Scene.jsx` creates `positionRef` once with `useRef` and passes the same ref object to both `Player` and `FollowCamera`. Player writes to `positionRef.current` inside its `useFrame`; FollowCamera reads it inside its own `useFrame`. Because both hold a reference to the same object, they see the same values with zero React involvement. This is a shared mutable channel — used deliberately here because it is the hot path.

Q What is the dependency array in `useEffect` and where did it matter?

- A** It tells React when to re-run the effect. Two places mattered. The keyboard listener effect has `[]` so listeners are attached once on mount and removed on unmount — re-attaching on every render would leak listeners. The player-reset effect depends on `[maze, positionRef]`, so when the level changes and a new maze object is created, the player teleports back to the new start position. If I had left that array empty, starting Level 2 would leave the player at Level 1's coordinates, possibly inside a wall.

Q Why `useCallback` on the handlers passed into the 3D scene?

A Without it, `handleEnterNode` and friends would be brand new function objects on every render of `App`. Since they are passed as props into `Scene` and down to `Player`, that would make those props change identity constantly, defeating any memoisation and re-registering the `useFrame` callback more often than necessary. `useCallback` keeps the identity stable as long as the real dependencies have not changed.

Q Why did you use `React.StrictMode`? Did it cause problems?

A It is in `main.jsx` because it surfaces unsafe patterns in development by double-invoking renders and effects. It is important to know that it double-invokes *render and effects*, not `useFrame` callbacks, so it does not double the game speed. It did make me careful that effects are idempotent — for example, the player-reset effect just assigns a position, so running it twice is harmless.

Q How do you handle cleanup?

A Every effect that subscribes returns a cleanup function. The controls hook removes its `keydown`, `keyup` and `blur` listeners; the scenario popup and feedback card remove their keyboard shortcut listeners when they unmount, which also prevents the number keys from answering a scenario that is no longer on screen.

Q Why is there a `<Suspense>` inside the Canvas?

A React Three Fiber uses suspense for anything that loads asynchronously — textures, models, some drei helpers. Without a Suspense boundary, a component that suspends throws and takes the canvas down. I do not load external assets, but the boundary costs nothing and makes the scene safe to extend later.

6. Interview Q&A — State Management

Q Walk me through your state shape.

A

```
{
  phase,           // which screen: START, PLAYING, SCENARIO, FEEDBACK, ...
  level,           // 1, 2 or 3
  scores,          // { wealth, security, lifestyle } across the whole game
  levelScores,     // same three, reset at the start of each level
  balance,         // cash in rupees
  decisions: [],   // full history, one entry per choice made
  completedScenarios: [], // ids, so a node only pops up once
  activeScenario,  // the scenario currently on screen
  lastChoice,      // { scenario, option } driving the feedback card
  highestLevelUnlocked,
  muted
}
```

Two score objects exist because the level-complete screen grades only that level, while the final report grades the whole game. `decisions` keeps the full history so the final report can list every mistake and explain it again.

Q Why useReducer instead of several useStates?

A Because one user action changes many fields at once and they must change together. Choosing an option updates three scores, the level scores, the balance, the completed list, the decision history, the last choice, and the phase — seven fields in one transition. With separate `useState` calls that is seven setters that could interleave or be partially applied, and the "what changes when" logic gets scattered across components. With a reducer it is one pure function, one place to look, and it is trivially testable because it is just `(state, action) => state`.

Q Why not Redux, Zustand, or Jotai?

A Scope. There is one store, eight action types, no async work, no server cache, no middleware, no time-travel requirement. Context plus useReducer is built into React and costs zero bytes of bundle. Adding Redux Toolkit would add dependency weight and boilerplate for no functional gain. I would reach for Zustand or Redux if the state grew to multiple independent slices, or if Context re-renders became a measured bottleneck.

Q Isn't Context slow? Every consumer re-renders when the value changes.

A Yes, and that is a real limitation I am aware of. The reason it is not a problem here is that the value only changes on discrete user actions — starting a level, opening a scenario, choosing an option. That is a handful of updates per minute, not per frame. The per-frame work deliberately bypasses Context entirely and runs through refs inside `useFrame`. If I did put the player position in Context, every consumer would re-render 60 times a second and the game would stutter. Splitting the context or memoising the provider value would be the next step if the tree grew.

Q How does saving work? Why localStorage?

A A `useEffect` in the provider runs after every state change and writes a subset of state to `localStorage` under the key `financial-maze-3d:save:v1`. On load, `loadState()` reads it back. `localStorage` was the right choice because the brief called for no backend: it is synchronous, needs no permissions, and survives a tab close. The version suffix in the key means I can change the state shape later without old saves corrupting a new build.

Q What can go wrong with localStorage, and how did you handle it?

A Three things. It can be unavailable or throw in private browsing mode; it can be full; and it can contain stale data from an older version of the game. Both the read and the write are wrapped in `try/catch`, so any failure degrades to "the game works, it just will not resume" rather than a white screen. On load I also spread the saved data over a freshly created default state, so a missing field in an old save is filled in rather than becoming `undefined`.

Q Why does loading a save always return you to the start screen?

A Because restoring straight into a transient phase would be broken. If you closed the tab while a decision popup was open, restoring `phase: SCENARIO` would show a popup for a scenario the 3D scene has not positioned the player next to. So `loadState` explicitly forces `phase` back to `START` and clears `activeScenario` and `lastChoice`. Persist durable facts, not transient UI state.

Q Is your reducer pure? Why does that matter?

A Yes — it never mutates the incoming state, it spreads into new objects and arrays, and it has no side effects. That matters because React may call a reducer more than once for the same action (in `StrictMode` it deliberately does), and impure reducers would then double-apply score changes. It also makes the reducer testable in isolation, which is how I verified a full 18-scenario playthrough without a browser.

Q How does the game know the exit should unlock?

A A derived helper, `areAllNodesDone(state)`, checks that every scenario for the current level appears in `completedScenarios`. It is computed on render rather than stored in state, because storing it would mean two sources of truth that could drift out of sync. Deriving is cheap here — it is six array lookups.

7. Interview Q&A — React Three Fiber & 3D Graphics

Q What is React Three Fiber and why use it over plain Three.js?

A React Three Fiber is a React renderer for Three.js. Instead of imperatively creating a scene, camera, geometry and material and calling `scene.add()`, you describe the scene declaratively as JSX — `<mesh>`, `<boxGeometry>`, `<meshStandardMaterial>` — and it reconciles that into real Three.js objects. The benefit is that the 3D scene participates in React's component model: I can map over my maze nodes with `.map()`, conditionally render the exit portal as locked or unlocked, and pass props down, all with normal React thinking. It is not a wrapper or an abstraction over Three.js — the underlying objects are plain Three.js, so anything in the Three.js docs still applies.

Q Explain `useFrame` .

A `useFrame` registers a callback that runs on every frame of the render loop, before the frame is drawn. It receives the R3F state (clock, camera, scene, renderer) and `delta`, the seconds elapsed since the previous frame. It is where all per-frame animation lives: moving the player, pulsing the orbs, easing the camera. The important discipline is that you must not call React setters inside it — you mutate Three.js objects and refs directly.

Q Why multiply by `delta` ?

A To make movement frame-rate independent. If I moved the player by a fixed amount per frame, it would travel twice as fast on a 120Hz monitor as on a 60Hz one. Multiplying speed by `delta` converts "units per frame" into "units per second", so the player moves at 9 world units per second regardless of frame rate.

Q You cap delta with `Math.min(delta, 0.05)` . Why?

A If the browser tab is backgrounded or the machine stalls, the next frame's `delta` can be several seconds. Multiplied by speed, that is a single movement step of dozens of world units, which would teleport the player straight through a wall — classic tunnelling, because collision is tested at the destination, not along the path. Capping delta at 50ms means the worst-case single step is 0.45 units, far smaller than the 0.9 player radius, so the collision test can never be skipped over.

Interestingly, this cap is also what made the game look frozen when I tested it in a background tab: the browser throttled `requestAnimationFrame` to roughly one frame a second, and each of those rare frames only advanced 0.05 seconds of movement. That was correct behaviour, not a bug.

Q What is an instanced mesh and why did you need one?

A A Level 3 maze has over 200 wall blocks. Rendering each as its own `<mesh>` means 200 separate draw calls per frame, and draw calls are the expensive part — each one is a CPU-to-GPU handoff. An `<instancedMesh>` uploads one geometry and one material once, plus an array of transformation matrices, and the GPU draws all 200 copies in a single call. I set each instance's matrix with `setMatrixAt(i, matrix)` and then must set `instanceMatrix.needsUpdate = true` so Three.js re-uploads the buffer. I use three instanced meshes for the walls — body, glowing cap, and dark inlay — so the whole maze is 3 draw calls instead of 600.

Q How did you get the Tron-style glowing wall edges?

A This is my favourite detail because the first attempt looked wrong. Each wall is three stacked layers: a dark body, then a bright plate covering the whole top, then a slightly *smaller dark plate* on top of that. The small dark plate masks the middle of the bright plate and leaves only a thin bright border showing around the edge. Because neighbouring walls sit flush, those borders join into continuous neon lines running along the top of every corridor.

My first version was just a bright plate with no inlay, and the result was enormous glowing cyan slabs — it read as a lit platform, not an outline. The masking trick was the fix, and it is far cheaper than post-processing bloom.

Q Why not use a bloom post-processing effect for the glow?

A Bloom means adding `@react-three/postprocessing`, running an extra full-screen render pass every frame, and a noticeably larger bundle. I got a convincing neon look with emissive materials, `toneMapped={false}` so bright colours are not compressed by tone mapping, and point lights at the decision nodes. That keeps the frame budget and the bundle small, which mattered because three.js already dominates the download.

Q How does the follow camera work?

A Each frame it computes where the camera ideally should be — the player's position plus a fixed offset of 30 units up and 24 back — and then moves a fraction of the way there rather than snapping. The fraction uses exponential smoothing: `alpha = 1 - Math.exp(-k * dt)`. That formula matters: a naive `lerp(target, 0.1)` would ease faster on a 144Hz monitor than on a 60Hz one, because it runs more often. The exponential form makes the easing identical at any frame rate. When a popup is open, the camera switches to slowly orbiting the player, which keeps the background alive without pulling attention off the text.

Q What lighting did you use?

A A dim blue `ambientLight` so nothing is pure black, a cool `directionalLight` as the key from above, a warm one from the opposite side for rim lighting, and a `hemisphereLight` for sky-to-ground gradient. Then each decision node and the player carry their own `pointLight`, so the orbs genuinely illuminate the walls around them rather than just appearing bright. Most of the visual punch comes from emissive materials rather than lights, which is cheaper.

Q What is fog doing in the scene?

A Two jobs. Visually, it fades distant geometry into the background colour so the maze feels like it sits in endless dark space instead of ending at a visible edge. The fog colour is set to exactly the background colour, so the transition is invisible. Practically, it also hides the far edge of the floor plane so I do not need an enormous floor.

Q Why is the floor plane rotated by `-Math.PI / 2`?

A Because `planeGeometry` in Three.js is created standing upright in the XY plane, facing the camera. To make it a floor it has to be rotated -90 degrees about the X axis so it lies flat in the XZ plane. It is one of those constants you write once and then always recognise in Three.js code.

Q How are the decision node animations done?

A Inside `useFrame`, driven by `state.clock.elapsedTime`. The orb bobs with a sine wave, the halo pulses by scaling with another sine, and 14 particles orbit it using an instanced mesh where each particle's angle is `(i / count) * 2π + t * speed`. Each node adds a phase offset based on its index so the six orbs do not all pulse in unison, which would look mechanical. Answered nodes switch from gold to green and stop pulsing, giving the player a readable map of what is left to do.

Q How do you handle window resizing?

A React Three Fiber's `<Canvas>` handles it automatically — it observes its container and updates the renderer size and the camera's aspect ratio. That is one of the concrete conveniences over hand-rolled Three.js, where you write a resize listener that calls `camera.updateProjectionMatrix()` yourself. On the CSS side the canvas is `position: absolute; inset: 0`, and the UI overlay is responsive with a mobile breakpoint at 860px.

Q What is `dpr={1, 2}` on the Canvas?

A Device pixel ratio, given as a range. On a high-DPI screen, rendering at the full native ratio (3 or 4 on some phones) means shading many times more pixels for very little visual gain. Clamping it between 1 and 2 keeps the render sharp on retina displays while capping the worst case, which is a cheap and effective performance guard on mobile.

8. Interview Q&A — Game Logic, Maze & Collision

Q How is a maze represented?

A As text art — an array of strings, one per row, in `data/levels.js`:

```
#####  
"#S#.#...#...#6..#"  
"#.#.#.#.#.#E#.#.#"
```

is a wall, . is floor, S is the start, E is the exit, and the digits 1 – 6 are decision nodes where the digit selects which scenario appears. `utils/maze.js` parses that into wall positions, node positions and a boolean lookup grid for collision. The payoff is that a whole level can be redesigned by editing strings, with no changes to any 3D code, and the layout is readable in the source.

Q How do you convert grid coordinates to 3D world coordinates?

A $x = (\text{col} - \text{centreCol}) * \text{CELL}$ and $z = (\text{row} - \text{centreRow}) * \text{CELL}$, with `CELL = 4`. Subtracting the centre means the maze is centred on the origin, which keeps camera maths simple and avoids large coordinate values. The inverse, `worldToCell`, rounds the reverse calculation to get back to a row and column.

Q Explain your collision detection.

A It is circle-versus-axis-aligned-box. The player is a circle of radius 0.9 on the floor plane; every wall is a 4×4 box. For a candidate position I find the player's grid cell, then check only the 3×3 block of cells around it — since a cell is 4 units and the player radius is under 1, no wall outside that neighbourhood can possibly be touching. For each solid cell I clamp the player's centre to the wall box to find the nearest point on that box, then compare the squared distance to the squared radius. Using squared distance avoids a square root in the inner loop.

Checking a fixed 3×3 neighbourhood rather than all 200 walls is what makes this $O(1)$ per frame instead of $O(n)$.

Q How does the player slide along walls instead of getting stuck?

A By testing each axis separately rather than the combined movement vector:

```
if (dx !== 0 && canStandAt(maze, x + dx, z)) nx = x + dx;  
if (dz !== 0 && canStandAt(maze, nx, z + dz)) nz = z + dz;
```

If you are pushing diagonally into a wall, the blocked axis is rejected but the free axis still applies, so you glide along the surface. If I tested the combined vector in one go, any diagonal contact would stop movement completely and the player would feel like they were catching on every corner. Note the second test uses `nx`, the already-updated X, so the two axes compose correctly.

Q How does the game detect that you have reached a decision node?

A Two tests combined with OR: the player is in the same grid cell as the node, or within 1.4 world units of its centre. I originally had only the radius check, and it was unreliable — when the player slides along a wall they sit off-centre in the corridor, and the geometry works out that they can clip the corner of a node's cell while staying just outside 1.4 units. So you would walk visually over an orb and nothing would happen. Adding the exact cell match makes stepping onto an orb always register, and the radius still catches near-misses that feel like a hit. The same pattern is used for the exit portal.

Q How do you stop the same popup firing every frame while standing on a node?

A A ref, `insideNodeRef`, remembers which node the player is currently overlapping. The popup only dispatches when the touched node differs from the remembered one, and the ref is cleared when the player is not touching any node. Without that, `useFrame` would dispatch `OPEN_SCENARIO` 60 times a second. The reducer also guards independently by ignoring scenarios already in `completedScenarios` — two layers, because the reducer should be safe regardless of who calls it.

Q How were the mazes generated?

A I hand-drew them first and that was a mistake — several had holes in the outer wall and nodes walled off in unreachable pockets. So I wrote a generator: a randomised depth-first search (recursive backtracker) over a cell grid, which produces a "perfect" maze where every cell is reachable and there are no loops. Then I knock out a controlled number of interior walls to add loops, because a perfect maze is all dead ends and is frustrating to walk. The random number generator is a seeded mulberry32, so the same seed always produces the same maze and the levels are reproducible.

Node placement then runs a BFS from the start to get the path distance to every cell, and picks six cells spread evenly across the distance range with a minimum separation, so the decisions are met progressively rather than clustered. The exit is the farthest reachable cell that is also far in straight-line terms, so it never ends up next to the spawn behind a single wall.

Q How do you know the mazes are actually solvable?

A I wrote a validator that runs against the real modules and checks each level: that the grid is rectangular, that the entire border is wall, that there is exactly one start and one exit, that there are six correctly numbered nodes, and that a BFS from the start reaches every node and the exit. Then a second, stronger test actually *walks* the player through the maze using the real `moveWithCollision` code along BFS paths and asserts that all six nodes trigger. That second test is the meaningful one, because it exercises collision and detection, not just connectivity.

Q How is the grade calculated?

A `getMaxScoresForLevel()` walks that level's scenarios and sums the best available value on each axis, giving the maximum score obtainable. The player's total is divided by that maximum to get a percentage, which maps to a letter: A+ at 85%, A at 70, B at 55, C at 40, D below. The key property is that the maximum is derived from the data rather than hard-coded, so if I add or edit a scenario the grading stays correct automatically with no other file touched.

Q How does the final report projection work?

A It maps the player's Wealth score onto a plausible monthly investment — a maxed Wealth score implies about ₹25,000/month by 30, a zero score implies almost nothing — and then runs a standard future value of an annuity-due calculation at 12% a year with monthly compounding: $FV = P \times ((1 + r)^n - 1) / r \times (1 + r)$, where r is the monthly rate and n the number of months. I sanity-checked it against the claims made in the scenario text: ₹15,000/month for 17 years comes out at ₹1.00 crore, and ₹2,000/month for 30 years at ₹70.6 lakh, which are exactly the figures the scenarios quote.

9. Interview Q&A — Performance & Optimisation

Q What did you do for performance?

- A
- **Instanced meshes** — the whole maze renders in 3 draw calls rather than 600.
 - **Refs instead of state** in the per-frame path, so React does zero work during animation.
 - **useMemo on the parsed maze**, so grid parsing happens once per level rather than on every render.
 - **O(1) collision** by only checking the 3×3 cell neighbourhood.
 - **Squared distances** in the collision inner loop to avoid square roots.
 - **Capped device pixel ratio** at 2.
 - **CSS transitions for the score bars** instead of animating in JavaScript, so the browser compositor does the work.
 - **Manual chunk splitting** so three.js caches separately from app code.

Q What is the bundle size and are you happy with it?

A Gzipped: about 176 KB for three.js, 45 KB for React, 68 KB for my application code and 4 KB of CSS. three.js dominates, and roughly 175 KB is the practical floor for any real 3D web app — it is the cost of doing 3D at all. I split it into its own chunk with Rollup's `manualChunks` because three.js changes rarely, so a returning visitor keeps it cached while my game code updates. I also avoided extra libraries: no Howler for audio, no postprocessing, no animation library, no state library.

Q How would you reduce it further?

A Lazy-load the Canvas and the three.js chunk with `React.lazy` so the start screen paints before the 3D engine downloads — that would make first paint dramatically faster. Beyond that, a custom three.js build that tree-shakes unused renderers and loaders, and dropping drei by hand-writing the two helpers I use from it (Grid and Stars).

Q How would you profile this if it were slow?

A Chrome DevTools Performance tab to see whether frames are CPU-bound or GPU-bound, and `renderer.info` from Three.js to read the actual draw call and triangle counts — that tells you immediately whether the problem is draw calls, geometry, or shading. React DevTools Profiler with "highlight updates" would show whether something is re-rendering per frame that should not be, which is the classic R3F mistake.

Q What would break if a maze were ten times larger?

A Rendering would largely be fine because of instancing. Collision would still be $O(1)$ thanks to the neighbourhood check. The real pressures would be the fixed camera height showing too little of a huge maze, and the fact that I render every node and its point light regardless of distance — I would need to cull lights and nodes outside the camera view, since point lights are one of the more expensive things per fragment.

10. Interview Q&A — Audio, Input & Accessibility

Q How does the audio work with no sound files?

A Everything is synthesised at runtime with the Web Audio API. A helper creates an `OscillatorNode` and a `GainNode`, sets a frequency and waveform, and shapes the volume with a quick attack and an exponential decay. All the game sounds are built from that: the "good decision" sound is a rising three-note arpeggio, the warning is a descending sawtooth pair. The ambient pad is two detuned oscillators — a 55 Hz sine and an 82.5 Hz triangle — through a low-pass filter, with a very slow 0.07 Hz LFO modulating the filter cutoff so it breathes instead of sitting static.

Q Why exponential ramps on the gain rather than just stopping the oscillator?

A Because cutting a waveform off abruptly produces an audible click — a discontinuity in the signal is broadband noise. Ramping the gain down smoothly avoids it. It also has to be `exponentialRampToValueAtTime` rather than linear, and the value can never be exactly zero, which is why the ramps target 0.0001 — exponential ramps to zero are undefined.

Q Browsers block autoplaying audio. How did you handle that?

A Modern browsers create the `AudioContext` in a "suspended" state until a genuine user gesture. So I do not create the context on mount. Instead there is a one-time listener for the first `pointerdown` or `keydown`, which calls `ctx.resume()` and starts the ambient pad, then removes itself. That is the standard unlock pattern.

Q How does mute work?

A It ramps the master gain node down rather than stopping any oscillators. The ambient oscillators keep running silently, so unmuting is instant and the pad does not restart mid-phrase. Stopping and recreating oscillators would be both slower and audibly wrong, since an `OscillatorNode` cannot be restarted once stopped.

Q Why is the held-key state in a ref rather than state?

A Same reason as the player position: the game loop reads it 60 times a second. If holding W set React state, every keypress would re-render the tree. The listeners write booleans into a ref, and `getDirection()` reads that ref inside `useFrame`.

Q Why do you normalise the movement vector?

A Because holding W and D together produces a vector of length $\sqrt{2}$, which would make diagonal movement about 41% faster than straight movement — a very common bug in beginner game code. If the combined length exceeds 1, I divide both components by it, clamping the magnitude to 1 so speed is the same in every direction.

Q Why do you clear all keys on window blur?

A If the player holds W and then alt-tabs away, the browser never delivers the `keyup`, so the key would stay logically pressed and the player would keep walking forever when they came back. Listening for `blur` and resetting the key state fixes that. Amusingly, this handler is what made my automated browser testing look broken — the test tool's own interactions blurred the page and cleared the held keys — but the behaviour is correct for real users.

Q How does mobile support work?

- A** An on-screen joystick rendered only on touch devices. It uses Pointer Events rather than Touch Events, so the same code path works with a mouse, which makes it testable on a laptop. Dragging the thumb produces a vector that is clamped to the ring radius and normalised to $-1..1$, then fed into the same `getDirection()` as the keyboard, so there is one movement code path rather than two.

Detecting touch is done with `window.matchMedia("(pointer: coarse)")`. I originally used `navigator.maxTouchPoints > 0`, but that is true on any touchscreen laptop, so the joystick appeared for desktop users driving with a mouse. The media query asks whether the *primary* pointer is coarse, which is the right question.

Q What accessibility considerations did you make?

- A** The entire game is playable from the keyboard: WASD or arrows to move, number keys 1–4 to answer a scenario, Enter or Space to dismiss the feedback card. Buttons have `aria-label`s, the glossary panel is marked `aria-hidden` when closed, and text is white or gold on a very dark background for contrast.

I would not claim it is fully accessible, and I can say honestly what is missing: there is no screen reader path for the 3D maze itself, no focus trap inside the modal cards, and no reduced-motion mode for users who are sensitive to the pulsing and camera motion. Those would be my first accessibility tasks.

11. Interview Q&A — Build, Tooling & Deployment

Q Why Vite instead of Create React App?

- A** CRA is deprecated and no longer recommended by the React team, and it fails on current Node versions with React 18/19. Vite uses native ES modules in development, so the dev server starts in under half a second and hot updates are near-instant regardless of project size, where CRA rebuilds a webpack bundle. For production it uses Rollup, which gives better tree-shaking and easy chunk control. I kept an `npm start` script aliased to `vite` so the familiar command still works.

Q Explain the manualChunks configuration.

- A** By default Rollup bundles everything into one file, which triggered a warning at over 1 MB. I split `three.js` and `React` into their own chunks. The reasoning is caching: `three.js` is 683 KB raw and essentially never changes, so on a redeploy the browser only re-downloads my ~209 KB application chunk and keeps the rest from cache. It also lets the browser fetch the chunks in parallel.

Q What is hot module replacement, and did it cause you trouble?

- A** HMR swaps a changed module into the running page without a reload, preserving state. It is excellent for UI work. It is genuinely awkward with WebGL, though: each reload cycle can leave the previous renderer's context behind, and browsers cap the number of live WebGL contexts at around 16. During development I hit that cap after many rapid edits and the canvas went blank with no error in the console — everything else still worked, which made it confusing. A hard reload in a fresh tab fixed it. The lesson I took is that a blank canvas with no exception usually means a lost or exhausted context, not broken code.

Q How would you deploy this?

A `npm run build` produces a fully static `dist/` folder, so any static host works — Netlify, Vercel, GitHub Pages, S3 with CloudFront. There is no server component, no environment variables and no secrets. For GitHub Pages I would need to set Vite's `base` option to the repository name so asset paths resolve under the subpath.

Q What is in your `.gitignore` and why?

A `node_modules` (reinstallable from the lockfile, and huge), `dist` (build output, regenerable), logs, editor folders like `.vscode` and `.idea`, OS files like `Thumbs.db`, and `.env` files. The principle is that anything generated, machine-specific, or secret stays out of version control. `package-lock.json` is committed, because it pins exact dependency versions so a teammate's install matches mine.

Q Why JavaScript and not TypeScript?

A It was a project constraint. I will say honestly that TypeScript would help here — the scenario objects have a fixed shape with an enum-like `verdict` field, and a typo in `impact.wellth` would currently fail silently and produce `NaN` scores. I partly compensated by writing a validation script that checks every scenario has the right fields, valid verdicts, in-range impacts and a non-trivial explanation. That is a runtime approximation of what types would give at compile time.

12. Interview Q&A — Testing & Debugging (Real Bugs I Fixed)

These are strong interview stories because each has a clear symptom, an investigation, a root cause and a fix. Interviewers value the reasoning far more than the outcome.

Q How did you test this? There is no test framework.

A I wrote Node scripts that import the real source modules and assert against them. Three suites: a maze validator, a walking simulation that drives the actual collision code along BFS paths and asserts every node triggers, and a content-and-rules suite that checks all 18 scenarios are well-formed and plays a full three-level game through the scoring logic.

I would use Vitest for a real project — it shares Vite's config and would let me test the reducer directly, plus React Testing Library for the UI. I did not add it here to keep dependencies minimal, and the plain Node scripts caught real bugs, which was the point.

Q Tell me about a bug you found and fixed.

A The best one was in the game's content, not the code. My test played a full game always choosing the best options and printed the balance at each level — and it went *negative*, ending at $-\text{₹}4,75,000$. Two causes. First, Level 3's home loan scenario said "You have ₹30 lakh saved" but offered an option costing ₹33 lakh, and two of its three options had the identical cost, which made them meaningless as a choice. Second, starting balances were simply too small for the amounts the later scenarios move.

I fixed the scenario to be internally consistent (₹35 lakh saved, with distinct ₹21L / ₹33L / ₹35L options), raised the Level 2 and 3 starting balances to what their own text assumes, and added a floor of zero in the reducer. The floor is not just papering over it — it is the honest model. If you cannot fund a flat in cash you simply cannot buy it, and draining to zero with the explanation attached is the correct lesson. I then added assertions that both a best-case and a worst-case player finish with a non-negative balance.

Q Tell me about a bug that turned out not to be a bug.

A I was testing in an automated browser and the player would barely move — a three second key hold advanced it about one cell instead of six. I spent a long time suspecting the collision code, the input handling, and stale closures. I eventually instrumented the page and found `document.visibilityState === "hidden"`.

The test browser window was never foregrounded, so Chrome throttled `requestAnimationFrame` to roughly one frame per second. My deliberate `Math.min(delta, 0.05)` cap then limited each of those rare frames to 50ms of movement, so the player crawled. Both behaviours were correct — the throttle is the browser saving battery, and the cap prevents tunnelling. The real lesson was about my debugging: I should have verified my measurement environment before doubting the code. I also confirmed the logic independently by simulating the walk in Node, which proved the movement and detection were right all along.

Q Tell me about a bug in your collision or detection code.

A Node detection originally used only a distance check of 1.4 units from the orb centre. It mostly worked, but occasionally you would walk visibly over an orb and nothing happened. The cause is that when the player slides along a wall they end up off-centre in the corridor — pressed against the wall rather than at the cell centre — and from that offset they can clip the corner of the node's cell while remaining just outside 1.4 units. It was a marginal case, which is why it was intermittent.

The fix was to add an exact grid-cell match alongside the radius check. The cell test is exact and cannot be squeezed past; the radius still catches near-misses that feel like a hit. I then wrote the walking simulation specifically so a regression here would fail a test rather than be noticed by a player.

Q Tell me about a visual bug.

A My first attempt at the neon walls put a bright emissive plate across the entire top of every wall block. Rendered, it looked nothing like the Tron reference — the tops merged into enormous glowing cyan slabs and read as lit platforms rather than glowing outlines, and the whole scene was blown out. The fix was the masking technique: keep the bright plate but cover its middle with a slightly smaller dark plate, leaving only a thin bright rim. Adjacent walls' rims join into continuous neon lines. I also pulled the camera back and reduced the emissive intensity. It was a good reminder that in graphics you have to look at the render — the code was doing exactly what I told it to.

Q How did you verify the UI screens if you could not play through in the test browser?

A I temporarily added a query-parameter hook to the state loader so I could open the app directly at `?preview=scenario`, `?preview=feedback`, `?preview=levelcomplete` or `?preview=report` with representative data. That let me screenshot every screen, verify the layout, and confirm the score deltas on the feedback card matched the data file exactly. Then I removed the hook and confirmed with a grep that no debug code remained before committing.

Q What would you add to a test suite if you had more time?

A Vitest unit tests for the reducer, covering every action type and the balance floor. Property-based tests for collision — generate random positions and assert the player is never inside a wall. React Testing Library tests asserting that pressing "2" selects the second option and that the feedback card shows the matching explanation. And Playwright for a real end-to-end playthrough in a visible browser, which would have avoided the background-tab throttling problem entirely.

13. Interview Q&A — Financial Domain Knowledge

If you present this project, expect domain questions — the content is half the work. Know these cold, and be ready to say "these are 2025–26 figures and rates change".

Q What is a SIP and why do you recommend it?

A A Systematic Investment Plan auto-invests a fixed amount into a mutual fund every month. Because the amount is fixed, it buys more units when prices are low and fewer when they are high — that is rupee cost averaging. It removes the need to time the market and, more importantly, removes emotion. ₹2,000 a month for 30 years at 12% compounds to roughly ₹70 lakh, which the game computes rather than asserts.

Q Why does the game insist on an emergency fund before investing?

A Because without a cash buffer, any shock — a medical bill, a layoff — forces you to either take high-interest debt or sell investments at whatever price the market happens to be offering, which is often during a downturn. The target is three to six months of expenses in a savings account or liquid fund. It is not an investment; its job is to stop a crisis becoming debt. Level 3 makes this concrete with a job-loss scenario where the fund buys you the ability to refuse a lowball offer.

Q Explain term insurance versus a ULIP or endowment plan.

A Term insurance is pure life cover with no investment component — if you die during the term your family gets the sum assured, otherwise you get nothing back, and that is exactly why it is cheap. ₹1 crore of cover costs about ₹12,000 a year at 25. A ULIP or endowment bundles insurance with investment and typically returns 4–6% while giving far less cover for the same premium. The principle is to never mix insurance with investment: buy pure term for protection, invest the difference in an index fund, and you end up with more of both. The premium is also locked at your entry age, so buying young is materially cheaper over a 30-year policy.

Q Why is refusing EPF a bad decision in the game?

A Because your employer matches your 12% contribution. Refusing it throws away a guaranteed 100% immediate return that is available nowhere else. EPF is also EEE — exempt at contribution, growth and withdrawal — and pays about 8.25% tax-free, which beats any fixed deposit. The game separately treats VPF (voluntary extra contribution) as only "okay" at a ₹25,000 salary, because it is good money locked up at exactly the stage when you most need liquidity.

Q Old versus new tax regime — how do you decide?

A The old regime has higher slab rates but allows deductions: 80C up to ₹1.5 lakh, 80D for health premiums, HRA, and up to ₹2 lakh of home loan interest under Section 24. The new regime has lower rates and removes almost all of them, but includes a ₹75,000 standard deduction. The crossover is roughly ₹3.5–4 lakh of total deductions: above that the old regime usually wins. A salaried person can switch every year, so the honest answer is to compute both. The game marks both as "good" answers for the right person, and marks "buying insurance purely to save tax" as bad — locking ₹1 lakh into a 5% endowment to save ₹30,000 of tax is a losing trade.

Q Why does the game treat credit cards as good *and* bad?

A Because the instrument is neutral and the behaviour is everything. Paying the full bill every month gives you up to 45 days of interest-free credit, rewards, and a rising CIBIL score, all free. Paying only the minimum compounds the balance at close to 40% a year and also forfeits the interest-free period on new purchases. The game also marks "reject the card entirely" as merely okay rather than good, because with no credit history you are invisible to lenders and your future home loan is harder and more expensive.

Q Why is buying gold jewellery marked as a bad investment?

A Because you lose 10–20% to making charges plus 3% GST on the way in, and lose the making charges again on the way out. That is a large guaranteed loss before any price movement. If you want gold exposure — and 10–20% of a portfolio in gold genuinely reduces volatility, since gold often rises when equity falls — hold it as a gold ETF or a Sovereign Gold Bond. Jewellery is consumption, and that is fine, it just is not an investment.

Q Where did the numbers come from, and how do you handle them going stale?

A They are typical Indian figures for 2025–26: EPF around 8.25%, PPF 7.1%, Sukanya Samriddhi 8.2%, home loans about 8.5%, long-run Indian equity 12–13%, gold 9–10%, education inflation 8%. Rates change, so there is a disclaimer on the final report stating the game is educational and not personalised advice, and recommending a SEBI-registered adviser. Practically, every number lives in one data file, so updating rates means editing `scenarios.js` and nothing else — which is exactly why I separated content from code.

Q Are you worried about giving financial advice?

A It is a real responsibility and I treated it as one. Three mitigations: the game teaches widely accepted principles rather than specific products — it never names a fund or an insurer; several scenarios deliberately have more than one correct answer, so it teaches reasoning rather than a script; and there is an explicit disclaimer directing users to a registered adviser. The framing throughout is "here is why this is usually true", not "do this".

14. Interview Q&A — Design Decisions & Trade-offs

Q Why does the exit stay locked until all decisions are answered?

A Because the maze is the delivery mechanism, not the point. If the exit were always open, a player optimising for completion would run straight to it and learn nothing. Locking it guarantees every player sees all six decisions, while still leaving them free to choose the order and route. The portal is visually red when locked and green when open, so the rule is communicated without a text prompt.

Q Why show the explanation after the choice rather than before?

A Because commitment is what makes the lesson stick. If the reasoning were visible up front, the player would just pattern-match to the highlighted answer and learn nothing. Choosing first, then finding out, produces a small moment of surprise when you were wrong — and that is the moment the information actually lands. It is the same reason the option buttons are colour-coded arbitrarily rather than by quality, so colour leaks no hint.

Q Why are there "okay" options and not just right and wrong?

A Because a binary right/wrong model would be dishonest about money. Sending your whole surplus to your parents is not a mistake — it is a real obligation that leaves you without a buffer. Declining to lend to a friend is financially correct but socially costly. Marking those as "okay" with an explanation of the trade-off teaches nuance, which is closer to how these decisions actually feel.

Q Why keep all content in one file rather than one file per scenario?

A Readability and reviewability. A single file can be read top to bottom in one sitting, which matters because the content is the part that most needs reviewing — by me, or by someone who knows finance but not React. Eighteen small files would mean eighteen imports and no way to scan the whole curriculum at once. If it grew to a few hundred scenarios I would split by level, but at this size one file is the right call.

Q Why a top-down camera instead of first-person?

A Orientation. First-person in a maze is disorienting and makes it hard to see where the remaining orbs are, which would turn the game into a navigation puzzle rather than a financial one. A raised follow camera lets you see the layout, spot the glowing pillars of light rising above the walls, and plan a route — so the maze stays pleasant traversal rather than the challenge itself.

Q Why do the decision node beams rise above the walls?

A So you can see where the remaining decisions are from anywhere in the maze. The walls would otherwise hide the orbs completely and you would be wandering blindly. The beams turn a potentially frustrating search into a navigation choice, and answered nodes dim and turn green so the player always has a readable map of progress.

Q Why does the balance floor at zero rather than showing negative?

A Because a negative cash balance is not a real thing — if you cannot fund a purchase you simply do not make it. Showing "–₹4,75,000" looks like a bug and undermines trust in the numbers. Draining to zero, paired with the explanation card saying the choice wiped out your savings, delivers the same lesson honestly. It was a deliberate modelling decision, not a patch over a crash.

Q If you rebuilt this from scratch, what would you change?

A TypeScript, for the reasons above — the scenario data has a strict shape and types would catch a whole class of silent errors. I would set up Vitest from day one rather than writing ad-hoc Node scripts. I would separate "invested" from "spent" in the money model, since right now buying an index fund and buying an iPhone both just reduce the balance, when one becomes an asset — tracking net worth alongside cash would be more truthful and would make the final report stronger. And I would lazy-load the 3D canvas so the start screen paints before three.js downloads.

15. Interview Q&A — Behavioural & "Tell Me About This Project"

Q What are you most proud of in this project?

A Two things. Technically, the wall-edge masking trick — three stacked instanced meshes that produce continuous neon outlines across the whole maze in three draw calls, after my first attempt looked completely wrong. Non-technically, the content: 18 scenarios where every single option has an honest explanation with real numbers I verified. It would have been much easier to write shallow filler, and the project would have been worthless.

Q What was the biggest mistake you made?

A Hand-drawing the mazes as text art and assuming they were fine. Several had holes in the outer wall and nodes sealed in unreachable pockets. I only found out because I wrote a validator. The deeper mistake was building on top of unverified foundations — I had already written the 3D rendering against those mazes before I checked they were solvable. Now I validate data structures before building on them, which is also why the walking simulation exists.

Q Describe a time you were stuck. How did you get unstuck?

A The movement bug I mentioned — the player crawling in the test browser. I cycled through several wrong hypotheses: collision, stale closures, the blur handler. What broke the loop was changing approach rather than pushing harder: I stopped trying to observe the running browser and instead reproduced the movement in Node against the real modules, which proved the logic was correct within a minute. That narrowed it to the environment, and instrumenting the page showed the tab was hidden and throttled. The lesson was to isolate the variable rather than keep re-testing the same uncertain setup.

Q How did you decide what to cut?

A I cut a bloom post-processing pass, background music files, and a "net worth versus cash" model. The criterion was whether it served the learning goal per unit of bundle size and complexity. Bloom would have cost a full-screen pass and a dependency for a marginal visual gain over emissive materials. The net worth model was the closest call — it is genuinely more truthful — and it is top of my list if I extend the project.

Q How would you extend this if you had another month?

A In priority order: a net worth model separating assets from cash; a fourth level on entrepreneurship and irregular income; the ability to replay a single scenario to compare outcomes; a shareable results card, since sharing is what would actually spread it; and accessibility work — a reduced-motion mode and a non-3D fallback so the content is reachable without WebGL.

Q What did you learn?

A Concretely: how a React renderer for a non-DOM target works, and why the "React builds the scene once, the loop mutates it" split matters. How to reason about draw calls. Why frame-rate independence needs exponential smoothing rather than a fixed lerp. More broadly, that verifying your assumptions early — the mazes, the balance arithmetic, the test environment — is cheaper than debugging on top of them later. Almost every bug I hit traced back to something I had assumed rather than checked.

Q Is this a solo project? What would change on a team?

A It is solo. On a team the content/code separation would pay off immediately — someone with finance expertise could own `scenarios.js` without touching React, and someone else could own the 3D layer. I would add TypeScript so those interfaces are enforced rather than assumed, set up CI running the validation suite on every pull request, and write down the maze text-art format as documentation rather than a file comment.

16. Known Limitations & Future Work

Being able to state your project's weaknesses clearly is a strong signal. It shows you evaluated your own work rather than just finishing it.

Current limitations

- **Money model is simplified.** Investing ₹2,000 in a SIP and spending ₹2,000 on dinner both just reduce the balance, when one becomes an asset. Net worth is not tracked.
- **No compounding during play.** Investments do not grow between levels; growth only appears in the final projection.
- **Requires WebGL.** There is no 2D fallback, so the content is unreachable on hardware without it.
- **Accessibility gaps.** No screen reader path for the maze, no focus trap in modals, no reduced-motion mode.
- **Fixed scenario order per level.** Node placement is deterministic, so replays are identical.
- **No automated test framework.** Verification is via Node scripts rather than Vitest, and there is no CI.
- **Rates will go stale.** They are 2025–26 figures and need periodic review.

Planned improvements

- Separate assets from cash and show a net worth figure.
- Lazy-load the 3D canvas so the start screen paints before three.js downloads.
- Vitest plus React Testing Library, with CI on pull requests.
- TypeScript, primarily to lock down the scenario data shape.
- Reduced-motion mode and a text-only fallback path.
- A shareable results card at the end.

17. Quick Revision Cheat Sheet

Numbers to remember

Levels / scenarios	3 levels, 18 scenarios, 6 per level, 3–4 options each
Maze sizes	17×17, 19×21, 23×23 cells; cell size 4 world units
Player	radius 0.9, speed 9 units/sec, delta capped at 0.05s
Camera	offset (0, 30, 24), fov 50, exponential smoothing $k = 3.2$
Node trigger	same grid cell OR within 1.4 units
Draw calls for maze	3 (instanced), not ~600
Bundle (gzip)	three 176 KB, React 45 KB, app 68 KB, CSS 4 KB
Grades	A+ ≥85%, A ≥70%, B ≥55%, C ≥40%, D below
Starting balances	L1 ₹50,000 · L2 ₹6,00,000 · L3 ₹35,00,000

One-line answers to the most likely questions

Question	Core answer
Why R3F?	Declarative JSX scene graph with React's component model, over real Three.js objects.
Why refs not state?	60 updates/sec would mean 60 full re-renders/sec.
Why instancing?	200 walls in 1 draw call instead of 200.
Why cap delta?	Prevents tunnelling through walls after a stall.
Why multiply by delta?	Frame-rate-independent movement.
Why per-axis collision?	Lets the player slide along walls instead of sticking.
Why useReducer?	One action changes seven fields atomically.
Why not Redux?	One store, 8 actions, no async — Context is enough and free.
Why useLayoutEffect in Maze?	Set instance matrices before paint, or walls flash at the origin.
Why Web Audio not Howler?	Synthesised sounds mean zero audio files and no extra dependency.
Why Vite not CRA?	CRA is deprecated and breaks on Node 24; Vite is faster and chunks better.
Why text-art mazes?	Redesign a level by editing strings, no 3D code changes.
Why lock the exit?	Stops players skipping the educational content.
Why explain after choosing?	Commitment first makes the correction memorable.
Why three scores?	Financial planning is a trade-off, not a single leaderboard.

The three stories to have ready

1. **The negative balance.** A test playthrough revealed content that did not add up; fixed the scenario, the starting balances, and added a modelled floor at zero.
2. **The crawling player.** Not a bug — a hidden tab throttling `requestAnimationFrame`, interacting with a deliberate delta cap. Solved by reproducing the logic in Node to isolate the variable.
3. **The glowing slabs.** The first neon wall attempt looked wrong; solved with a three-layer masking technique that costs nothing extra at render time.

Closing tip. When asked anything open-ended about this project, anchor on the three-layer architecture — content in `data/`, rules in the reducer, presentation split between the 3D scene and the 2D overlay. Almost every question maps cleanly onto one of those layers, and it shows you designed the structure rather than grew it accidentally.